

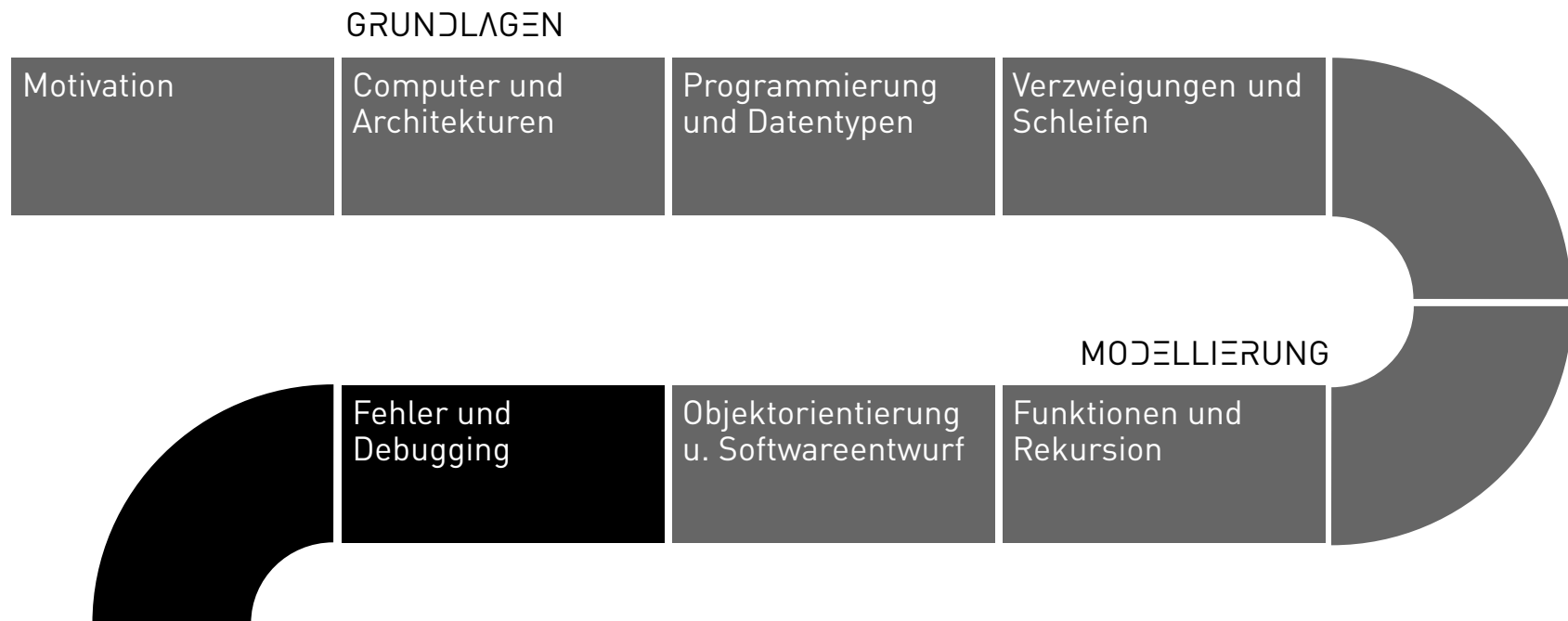
Programming and databases

Joern Ploennigs

Objects

MIDJOURNEY: BAUHAUS LINE DRAWING

PROCESS



OBJECTIVES

- Learn the fundamentals of object-oriented programming
- A basic understanding of programming with objects
- Data types vs. classes
- Functions vs. methods
- Introduction to advanced concepts such as inheritance

OBJECT-ORIENTED PROGRAMMING

Definition: Object-Oriented Programming

Object-oriented programming (OOP) is a programming paradigm that assumes that a program consists exclusively of objects that interact cooperatively with each other.

Each object has:

- *Attributes* (Properties): Define the value describing the state of an object.
- *Methods* define the possible state changes (actions) of an object.

WHY OBJECTS – THE SYNTACTIC PROBLEM OF REPEATING DATA STRUCTURES

- Objects are used to determine how repeating data structures are stored.
- As the program grows, so does the number of variables and data structures
 - for storing data
 - for controlling the flow of the program
 - for storing states
 - for processing input and output
- The underlying elements are usually based on repeating data structures.
- For example, point coordinates in blueprints or maps could be expressed as tuples or lists.

```
punkt_1 = (54.083336, 12.108811) # What is the correct syntax?  
punkt_2 = [12.108811, 54.083336]
```

WHY OBJECTS – THE SEMANTIC PROBLEM OF REPEATED DATA STRUCTURES

- Objects are also used to clearly define the semantics of values within a data structure.
- For instance, if we've agreed that we syntactically represent a point as a tuple, the meaning of the values is still unknown

```
punkt_1 = (54.083336, 12.108811) # What is the semantics of these values?  
punkt_2 = (12.108811, 54.083336) # So which is Latitude, which is Longitude?
```

WHY OBJECTS – THE BEHAVIORAL PROBLEM OF REPEATED DATA STRUCTURES

- Objects bundle functions and data structures.
- A distance function can be applied to the wrong data structures, e.g., to lines instead of points.

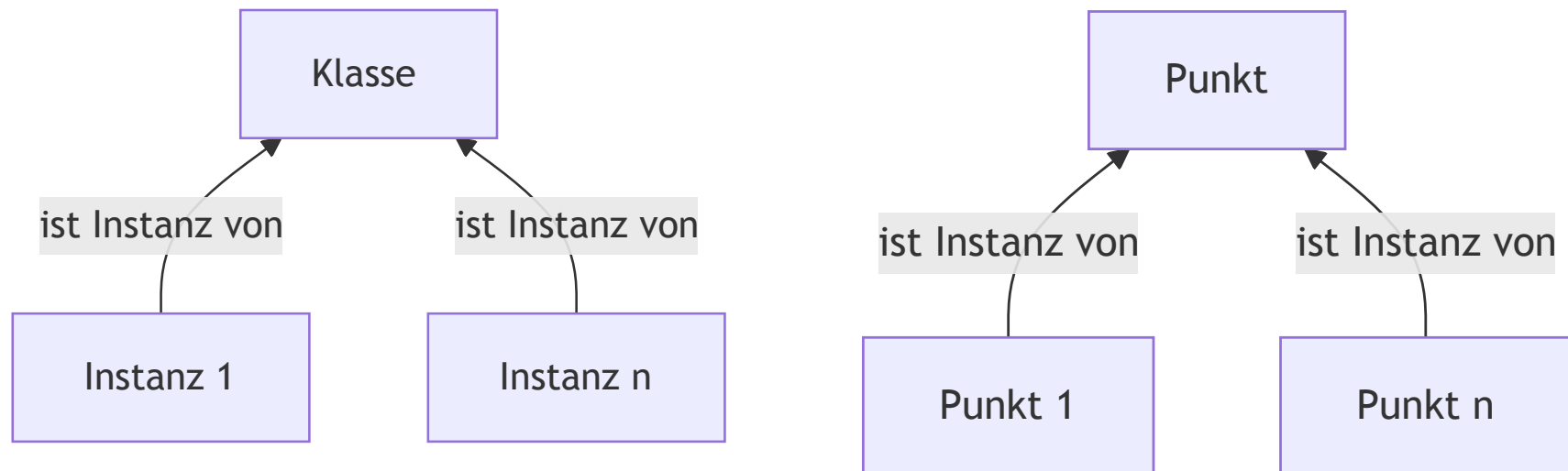
```
def distance(a, b):  
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)  
  
point_1 = (54.083336, 12.108811)  
point_2 = (12.108811, 54.083336)  
  
distance(point_1, point_2)  
  
line_1 = [(54.08, 12.11), (54.10, 12.11)]  
line_2 = [(12.11, 54.08), (12.20, 54.10)]  
  
distance(line_1, line_2)
```

BASICS OF OBJECT-ORIENTED PROGRAMMING

- Instead of using a large number of scattered data structures and functions in a confusing way, we group them into objects.
- The structure of these objects is defined in the form of *classes*, which act as a kind of blueprint. A class defines:
 - which *attributes* (variables / properties) an object of this class possesses.
 - which *methods* (functions) an object of the class provides

CLASSES AND INSTANCES

- Objects are *instances* of a class (the class is just a blueprint).
- A class can have any number of instances.
- All instances have the same structure.
- However, instances do not necessarily have the same values in their attributes.



OBJECT-ORIENTED PROGRAMMING IN PYTHON

- Python is object-oriented from the ground up – a fact we have so far ignored
- All data types in Python are objects (that's why they also have their own methods)
- User-defined classes are always a composite (complex) data type
 - Each data type has: value, type, identity
 - Variables are references

DEFINING CLASSES

The first step in creating an object is to define a new class for the object's type. This is done using the

`class` keyword:

```
class ClassName:  
    # class definition
```

CONSTRUCTORS WITH THE INIT METHOD

The constructor `__init__()` is a special method that defines how a new instance of the class is created.

```
class Point:
    # Constructor
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

- `self` describes a self-reference to the new instance of the class
- You can access the attributes of an instance using dot syntax
- `self.x` is therefore a reference to the instance's x attribute
- `self.x = x` means that we assign the value of the variable x to the new instance attribute x
- Since `__init__` is a function, you can also define defaults

CREATING INSTANCES

Instances of the class are created by using the class name as if it were a function (this implicitly calls the constructor). The `self` parameter is not supplied.

```
punkt_1 = Punkt(54.083336, 12.108811)
punkt_2 = Punkt(12.108811, 54.083336)
```

INSTANCE-LEVEL ATTRIBUTES

- Instance attributes can differ between instances
- If one instance changes the attribute, the change does not affect other instances (isolation)
- They are defined in the constructor `__init__`

```
class Punkt:  
    # Konstruktor  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

CLASS-LEVEL ATTRIBUTES

- Class attributes are attributes that have the same value for all instances of a class
- They are defined at the class level using the `class` keyword, just like a variable
- *Important!* They apply to all instances; if one instance changes the value, it changes for all other instances

```
class Punkt:  
    # Attribut aller Instanzen  
    einheit = "m"
```

METHODS

- Methods are defined like functions with the keyword `def`, indented at the same level as the class
- These methods are then available to all instances
- Methods always have `self` as the first parameter (reference to the current instance)
- This allows you to access attributes or other methods
- You can also assign defaults to method parameters

```
class Punkt:  
    # Methode  
    def distanz(self, punkt_2):  
        return math.sqrt((self.x - punkt_2.x)**2 + (self.y - punkt_2.y)**2)
```

CALLING METHODS

Class methods are available on all instances. They are invoked using dot syntax:

```
punkt_1.distanz()
```

In the same way, attributes can also be accessed:

```
print(punkt_1.x, punkt_1.y)
```

and these can be modified:

```
punkt_1.x = 20
```

Questions?

